Evaluating MCTS Enhancements and Spatial Heuristics in BattleSnakes

Dhanush Gowda Ramappa 5000343 [*1] **Roshun Sai Chandrasekaran 4981731** [*1] **Rahul Rao 4968964** [*1]

Abstract

This paper investigates the efficacy of non-neural Monte Carlo Tree Search (MCTS) enhancements compared to a deterministic spatial heuristic. We implement and evaluate a Vanilla MCTS against two advanced variants: one utilizing Rapid Action Value Estimation (RAVE) with Progressive Bias, and another employing Hierarchical Pruning to strictly filter lethal trajectories. Through multiple tournaments, we analyze agent performance using TrueSkill, ELO, and survival metrics. The results show that while aggressive pruning strategies can theoretically optimize search topology, a well-tuned, lightweight heuristic remains exceptionally competitive when hard latency limits hinder MCTS performance.

1. Introduction

BattleSnakes is a dynamic, multi-agent grid survival game. Four autonomous agents are tasked with maximizing their survival time and physical length while navigating a shrinking board populated by growing opponents and compounding environmental hazards. Recent advancements in battlesnake algorithms have been dominated by Deep Reinforcement Learning and neural network (NN) guided tree searches (Silver et al., 2016). However, deploying massive NNs requires specialized hardware for rapid inference. In standard environments bound by a strict 1 second timeout, the inference overhead of deep NNs bottlenecks the number of search iterations, making them computationally prohibitive. Hence, there is a necessity to explore the limitations and capabilities of traditional, non-neural search algorithms (Świechowski et al., 2022).

In this paper, we focus purely on heuristic driven and probabilistic search methodologies. We design a robust agent utilizing a time aware Breadth First Search (BFS) Voronoi heuristic to quantify territorial dominance. We subsequently use this spatial knowledge to guide the expansion and rollout phases of Monte Carlo Tree Search (MCTS). To mitigate the massive branching factor, we implement and evaluate standard Vanilla MCTS alongside two advanced modifications: Rapid Action Value Estimation (RAVE) combined with Progressive Bias, and Hierarchical Pruning with Proven States.

Through a rigorous parallelized tournament framework and fANOVA driven hyperparameter optimization, this research seeks to address the following Research Questions (RQs):

- **RQ1:** How do advanced MCTS modifications (RAVE, Progressive Bias, and action-space pruning) compare to a Vanilla MCTS baseline in a highly time-constrained, multi-agent environment?
- **RQ2:** Can a computationally lightweight, deterministic spatial heuristic outperform sampling-based MCTS approaches with server timeout constraints?

2. Environment Analysis & Assumptions

2.1. Overview and State Space

The BattleSnakes environment is a grid world where four agents compete to maximize survival time and length. Snakes start with three body segments and 100 health points, losing one health point per turn. Consuming randomly spawning food restores health to maximum and increases length, while moving out of bounds results in immediate elimination.

The game's combat and hazard mechanics force highly restrictive pathing. Head-on collisions eliminate the smaller snake (or both if equal), and any head-to-body or self-collision is instantly fatal. Hazard pits spawn sequentially from turn 26, applying compounding damage stacks to any snake entering head-first, effectively shrinking the safe navigable area. This simultaneous turn structure creates an extremely complex state space. With four agents acting concurrently, the joint action space features up to 256 (4^4) possible transitions per turn.

[*]Equal contribution. [1]LIACS, Leiden University, Netherlands. Correspondence to: Dhanush Gowda Ramappa <s5000343.@vuw.leidenuniv.nl>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

2.2. Assumptions and Forward Model Simplifications

Our forward model uses three main simplifications. First, to avoid the massive 4^4 branching factor of simultaneous turns, we only expand the search tree for our agent's moves. Opponent moves are instead approximated using a fast "paranoia" heuristic, assuming adversaries act rationally to survive and gather food. Second, we apply this assumption of rational opponents to the rollout phase. Completely random sampling makes weak positions look artificially survivable because simulated opponents often eliminate themselves. By limiting opponent moves to a logical subset, our simulations produce realistic outcomes without the heavy computation of deep adversarial modeling. We also simplified the hazard mechanics. Rather than tracking the exact lifespan and compounding damage of every hazard tile, we apply a static global damage penalty based on the current turn. This slight loss in accuracy drastically reduces memory use and speeds up state transitions, allowing for deeper MCTS iterations.

3. Methodology

The methodology systematically progresses from a deterministic baseline to advanced, heavily constrained probabilistic search trees. We first formulate a spatial heuristic that prioritizes territorial control and hazard avoidance. We then establish a standard Vanilla MCTS baseline. To accelerate search convergence within the strict 1000ms latency limit, we introduce two enhanced variations: MCTS Variant I integrates Rapid Action Value Estimation (RAVE) and Progressive Bias to rapidly identify promising branches, while MCTS Variant II implements strict Hierarchical Pruning and Proven State short-circuiting to mathematically eliminate fatal trajectories and optimize tree topology.

3.1. The Heuristic Agent

The foundation of our architecture relies on a deterministic spatial evaluation function designed to quantify the relative positional advantage of an agent within a given game state, denoted as $s \in S$. In a grid survival environments like BattleSnakes, survival probability is heavily correlated with spatial control (Perick et al., 2012). To formalize this, we calculate territorial dominance by partitioning the board into discrete regions. For a specific agent i, its controlled territory $T_i(s)$ is formally defined in Equation (1) as the set of all board tiles c that are strictly closer to our agent's head coordinate, $head_i$, than to any opponent j's head. The distance between two tiles is measured by the function $d(x, y)$:

$$T_i(s) = \left\{ c \in \text{Board} \mid d(head_i, c) < \min_{j \neq i} d(head_j, c) \right\} \tag{1}$$

Crucially, the distance function $d(x, y)$ is calculated using a time-aware BFS. This BFS treats opponent snake bodies as dynamic obstacles by calculating their exact lifetime (the number of turns until the tail vacates a specific tile). Furthermore, it evaluates dynamic hazard stacks; if an agent's health falls below the cumulative hazard damage threshold, hazard tiles are treated as impassable walls within the BFS. The raw heuristic score $H(s, i)$ in Equation (2) evaluates the current state by applying a set of tuned linear weights(w_1, w_2, and w_3) to three distinct metrics: our spatial dominance (Equation (1)), the proximity to the closest food item f from the set of all available food coordinates F, and the avoidance of danger. We define a Danger Zone, $D_i(s)$, as the set of all tiles immediately adjacent to the heads of opponent snakes that are larger than or equal to agent i. To enforce safety, we use an indicator function $\mathbb{I}(\cdot)$ that triggers a massive penalty if our head enters this lethal zone:

$$\begin{aligned} H(s, i) = w_1 |T_i(s)| + \frac{w_2}{\min_{f \in F} d(head_i, f)} \\ - w_3 \cdot \mathbb{I}(head_i \in D_i(s)) \end{aligned} \tag{2}$$

To integrate this heuristic smoothly into MCTS probabilistic equations later in the pipeline, we apply min-max normalization. By utilizing the theoretical minimum (H_{min}) and maximum (H_{max}) possible raw scores, we bind the heuristic between 0 and 1, producing our final normalized heuristic $\tilde{H}(s, i)$ as:

$$\tilde{H}(s, i) = \max\left(0, \min\left(1, \frac{H(s, i) - H_{min}}{H_{max} - H_{min}}\right)\right) \tag{3}$$

3.2. Vanilla Monte Carlo Tree Search

Our baseline agent implements standard MCTS. To balance the exploitation of known advantageous paths with the exploration of untried branches, the agent utilizes the Upper Confidence Bound applied to Trees (UCT) algorithm (Kocsis & Szepesvári, 2006).During the selection phase, the algorithm navigates the search tree from a parent node v to a child node v_a. Standard UCT evaluates each child using two distinct components: an exploitation term $Q_{MC}(v_a)$ in Equation (4) representing the average accumulated reward, and an exploration term $E(v_a)$ in Equation (5) that encourages visiting less-explored nodes. The exploitation term is calculated using $Q(v_a)$, the total simulated reward accumulated through the child node, divided by its total visit count $N(v_a)$. The exploration term scales with an exploration constant C and the natural logarithm of the parent node's visit count $N(v)$, divided by the child's visit count:

$$Q_{MC}(v_a) = \frac{Q(v_a)}{N(v_a)} \tag{4}$$

$$E(v_a) = C\sqrt{\frac{\ln N(v)}{N(v_a)}} \tag{5}$$

The agent selects the optimal action a^* from the set of available actions A_i that maximizes the sum of these two

terms (the UCB1 formula):

$$a^* = \arg\max_{a \in A_i} (Q_{MC}(v_a) + E(v_a)) \tag{6}$$

Once a leaf node is reached, the simulation phase executes a rollout policy $\pi_{sim}(s)$ that samples uniformly at random from a uniform distribution $\mathcal{U}$ over the set of physically legal, non-self-colliding moves A_{legal}, which is a subset of all possible actions A_i:

$$\pi_{sim}(s) \sim \mathcal{U}(A_{legal}) \tag{7}$$

Because these simulations are entirely random, they struggle to model opponent rationality, leading to high variance in terminal outcomes. Upon reaching a terminal state, the backpropagation phase updates all trajectory nodes with the terminal reward R, bounded between 0 and 1. This increments both the total reward $Q(v) \leftarrow Q(v) + R$ and the visit counts $N(v) \leftarrow N(v) + 1$ for every node along the path.

3.3. Monte Carlo Tree Search Variant I: Rapid Action Value Estimation and Progressive Bias

A pretty common limitation of standard UCT is its slow convergence in environments with high branching factors (Świechowski et al., 2022). To accelerate learning, our tunable variant modifies the baseline selection policy by integrating (Rapid Action Value Estimation) RAVE and a Progressive Bias term $B(v_a)$. First, RAVE tracks All-Moves-As-First (AMAF) statistics across the entire tree to create a fast, secondary exploitation term. We define the average AMAF reward for action a regardless of depth, denoted as $\bar{Q}_{AMAF}(a)$ in Equation (8). This is calculated by dividing its total accumulated AMAF reward $Q_{AMAF}(a)$ by its total AMAF visit count $N_{AMAF}(a)$:

$$\bar{Q}AMAF(a) = \frac{QAMAF(a)}{N_{AMAF}(a)} \tag{8}$$

Following (Rimmel et al., 2011), we create a blended exploitation term $Q_{blend}(v_a)$ that smoothly transitions trust from the AMAF estimate to the standard Monte Carlo estimate as the node's visit count grows. This transition is controlled by a blending coefficient $\beta(v_a)$, which relies on an equivalence parameter K that dictates how many visits are required before the standard Monte Carlo estimate outweighs the AMAF estimate:

$$\beta(v_a) = \sqrt{\frac{K}{3N(v_a) + K}} \tag{9}$$

$$\begin{aligned} Q_{blend}(v_a) = (1 - \beta(v_a))Q_{MC}(v_a) \\ + \beta(v_a)\bar{Q}_{AMAF}(a) \end{aligned} \tag{10}$$

Second, we guide the initial tree expansion using our spatial knowledge by introducing the Progressive Bias term $B(v_a)$. This mechanism injects our previously defined normalized territorial heuristic $\tilde{H}(s_{v_a}, i)$ in Equation (2) into the selection phase, scaled by a tunable progressive bias weight W_{PB} that aggressively decays as the node's visit count $N(v_a)$ increases:

$$B(v_a) = \frac{W_{PB} \cdot \tilde{H}(s_{v_a}, i)}{N(v_a) + 1} \tag{11}$$

By substituting the standard exploitation term (Equation (4)) with our blended calculation (Equation (10)) and adding the bias term (Equation (11)), the final selection policy for the enhanced variant selects the action a^* that maximizes the modular sum of exploitation, exploration, and spatial bias:

$$a^* = \arg\max_{a \in A_i} (Q_{blend}(v_a) + E(v_a) + B(v_a)) \tag{12}$$

3.4. Monte Carlo Tree Search Variant II: Hierarchical Pruning and Proven States

A second MCTS variant further optimizes the expansion phase tree topology to prevent the search from wasting computational resources on clearly fatal trajectories. We formalize this by evaluating $\mathbb{P}(\text{loss} \mid s, a)$, which represents the absolute probability of immediate elimination upon taking action a in state s. Using this probability, we partition the total action space A_i into mutually exclusive subsets: strictly safe moves A_{safe} with zero probability of death, risky moves A_{risky} with a fractional chance of death, and guaranteed fatalities A_{lethal} with a probability of one:

$$A_{safe} = \{a \in A_i \mid \mathbb{P}(\text{loss} \mid s, a) = 0\} \tag{13}$$

$$A_{risky} = \{a \in A_i \mid 0 < \mathbb{P}(\text{loss} \mid s, a) < 1\} \tag{14}$$

$$A_{lethal} = \{a \in A_i \mid \mathbb{P}(\text{loss} \mid s, a) = 1\} \tag{15}$$

The expansion policy restricts the searchable branches by enforcing a strict cascade. This guarantees the agent will update its available action space A_i to never explore a risky collision if a definitively safe physical move exists:

$$A_i \leftarrow \begin{cases} A_{safe} & \text{if } A_{safe} \neq \emptyset \\ A_{risky} & \text{if } A_{safe} = \emptyset \wedge A_{risky} \neq \emptyset \\ A_{lethal} & \text{otherwise} \end{cases} \tag{16}$$

It is vital to note that the spatial heuristic still actively guides this strictly pruned variant. Once hierarchical pruning isolates the safest subset of moves, the selection phase evaluates those moves using the progressive bias term $B(v_a)$. This ensures the search tree expands preferentially toward

paths that maximize territorial control.Finally, if a simulation mathematically guarantees elimination with a reward $R = 0$ or absolute survival with $R = 1$, the node is flagged as a proven state. For any proven node v_a, its value is permanently locked to this terminal value $R_{terminal}$ (Winands et al., 2008), meaning the average reward $Q(v_a)$ divided by $N(v_a)$ will mathematically yield $R_{terminal}$ indefinitely, causing further exploration of that branch to cease:

$$\forall N(v_a), \frac{Q(v_a)}{N(v_a)} = R_{terminal} \tag{17}$$

This short circuiting significantly increases simulation throughput, allowing the agent to reach significantly deeper evaluations within the given time limit.

4. Hyperparameter Tuning

After manual observation of initial gameplay, the MCTS variant II (incorporating hierarchical pruning) consistently demonstrated the strongest strategic potential. Consequently, we selected this variant for rigorous hyperparameter optimization using the Optuna framework (Akiba et al., 2019). To provide a demanding and dynamic baseline, the tuning agent played 50 trials (each consisting of 20 four-player matches) against our Heuristic, Vanilla MCTS, and MCTS Variant I agents. The optimization maximized an objective function prioritizing survival, length, and match victories:

$$\text{Score} = \left(0.8 \cdot \bar{S}_{surv} + 0.2 \cdot \bar{S}_{len}\right) + 0.2 \cdot W \tag{18}$$

Within Equation (18), $\bar{S}_{surv}$ represents the average survival score calculated as the number of turns survived divided by the 300-turn maximum, and $\bar{S}_{len}$ denotes the average normalized length growth scaled by a factor of 12. To explicitly reward victories, $W \in \{0, 1\}$ acts as a discrete binary bonus, equating to 1 if the agent wins the match and 0 otherwise.

To maximize this overall score, the search space encompassed the exploration constant ($C \in [0.5, 3.0]$), rollout depth limit (Depth $\in [6, 16]$), Progressive Bias weight ($W_{PB} \in [2.0, 15.0]$), and an early game target length ($L_{target} \in [8, 20]$).

To evaluate the optimization results, we utilized Functional Analysis of Variance (fANOVA), a statistical method that quantifies the fraction of performance variance attributable to each specific hyperparameter (Hutter et al., 2014). This analysis indicates that the exploration constant (C) is the most critical parameter, accounting for 47.3% of performance variance, followed by the Depth Limit (28.0%) and W_{PB} (19.9%) (Figure 1). Target Length proved largely insignificant (4.8%).

As illustrated in the Figure 2, the optimization distinctly favors lower C values. Because our hierarchical pruning

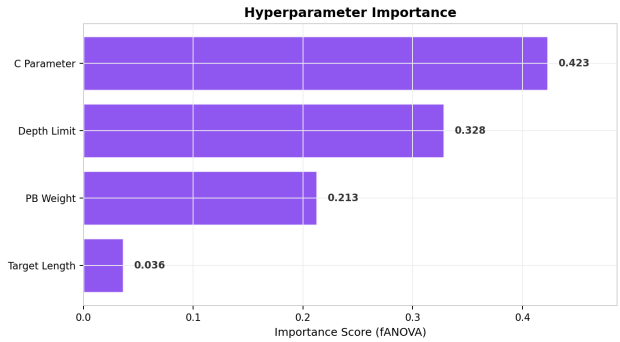

Figure 1. Hyperparameter importance using fANOVA for MCTS Variant 2

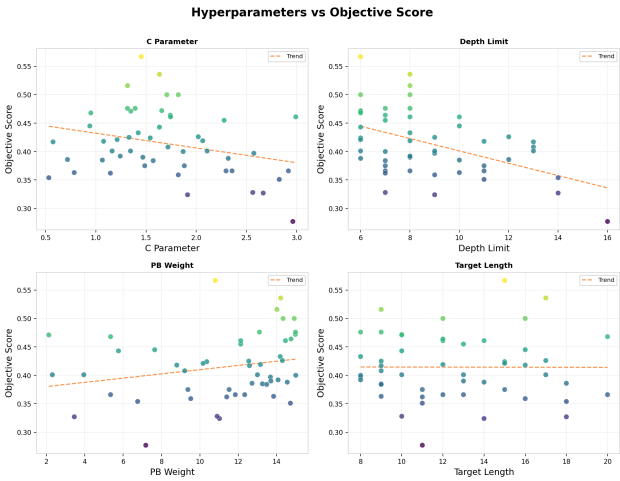

Figure 2. Hyperparameter values against the Objective Score, displaying performance trends.

already restricts the action space to strictly safe moves, the agent benefits heavily from exploiting known safe paths rather than exploring broadly. In contrast, shallower rollout depths were negatively correlated with performance. Within the server timeout limits, shallower rollouts execute significantly faster, allowing the MCTS to complete vastly more root-level iterations to establish a broad structural understanding of immediate physical threats. Conversely, higher Progressive Bias weights (W_{PB}) strongly correlate with better performance, confirming that aggressive spatial heuristic guidance is vital before sufficient Monte Carlo statistics can accumulate.

The list of final hyperparameter values for Vanilla MCTS and its advanced variants are detailed in Appendix B.

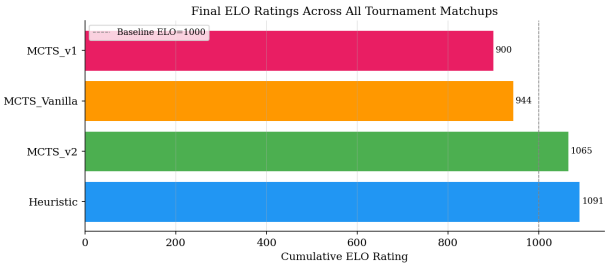

Figure 3. ELO ranking across all four agents

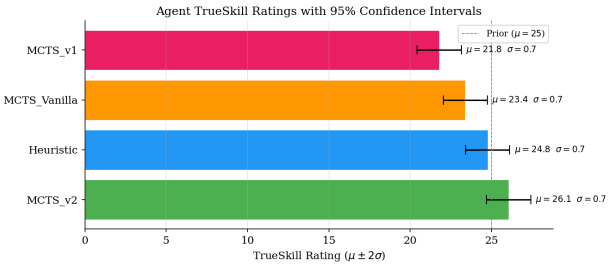

Figure 4. True skill ranking across all four agents

5. Tournament Results and Evaluation

To evaluate the agents, specifically referred to hereafter as Heuristic (the heuristic agent), MCTS Vanilla (Vanilla MCTS), MCTS_v1 (MCTS Variant I), and MCTS_v2 (MCTS Variant II), we conducted seven experiments: six pairwise matchups between agents and one all-four-agent experiment representing real contest conditions. Each pairwise experiment ran at least 25 games, while the four-agent experiment ran 50 games, resulting in 275 total games. Each game used a different random seed to ensure varied starting positions and food spawns, preventing repeated board configurations. All the game data was logged using the standard logger file. The skill ratings like ELO (Equation (19))was computed across all the experiments and TrueSkill was computed from the four-agent experiment only while pairwise experiments were used to calculate head-to-head win rates.

ELO Rating. ELO evaluates agents through a pairwise comaprison. Given ratings R_A and R_B, the expected score for agent A is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{19}$$

The ratings are updated as:

$$R'_A = R_A + k(S_A - E_A) \tag{20}$$

where $S_A \in \{0, 0.5, 1\}$ is the actual outcome (loss, draw, win) and $k = 32$ is the update factor. All agents start at ELO 1000. In a four-player game six pairwise comparisons are generated from the placement order. **TrueSkill.** TrueSkill is a Bayesian multiplayer rating system were each agent has a mean μ and uncertainty σ. All agents start with an $\mu = 25$ and $\sigma = 8.33$ for all agents. After each games both values are updated based on the complete placement order not just the winning results. The ranking score used for final comparison is:

$$score = \mu - 3\sigma \tag{21}$$

This lower confidence bound ensures an agent only ranks highly if it performs consistently well across many games.

The ELO ladder in Figure 3 ranked agents as: Heuristic (1091) > MCTS_v2 (1065) > MCTS_Vanilla (944) > MCTS_v1 (900). TrueSkill in Figure 4 produced a different ordering: MCTS_v2 ($\mu = 26.1$) > Heuristic ($\mu = 24.8$) > MCTS_Vanilla ($\mu = 23.4$) > MCTS_v1 ($\mu = 21.8$). The common pattern observed in both the plots is MCTS_v1 variation performing the weakest across the other agents.

The pairwise win rates (Figure 5) reveal distinct performance hierarchies across the seven experiments. In head-to-head matchups, the strictly pruned MCTS_v2 emerged as the dominant duelist, decisively defeating the Heuristic agent (60.0% in Exp 2), MCTS_Vanilla (65.6% in Exp 4), and MCTS_v1 (78.9% in Exp 6). Conversely, the unpruned MCTS_v1 struggled severely due to computational overhead, losing to both the Heuristic agent (65.5% in Exp 3) and the MCTS_Vanilla baseline (63.0% in Exp 5). However, in the chaotic four-player simulation representing real contest conditions (Exp 7), the Heuristic agent narrowly secured the lead with a 33.3% win rate compared to MCTS_v2's 30.9%. This final result suggests that under strict time constraints and massive branching factors, the immediate, robust spatial reactions of a fast heuristic can slightly edge out deeper multi-agent lookaheads. A prominent note on effects of complexity of agents is mentioned in Appendix A.2

6. Discussion

Two important results were observed from the tournament evaluations. The result that was most surprising was in Experiment 6 (Figure 5) were MCTS_v2 had a clear win percentage over MCTS_v1 which is actually more sophisticated. Battlesnakes is a game where sometimes a head-to-head gamble is the only surviving move, and sometimes approaching a larger snake's adjacent cell is the fastest way to claim a territory advantage before the opponent can. By completely pruning these moves from the action space, MCTS_v1 losses the ability to reason about risky but beneficial choices. It essentially never develops the skill to evaluate when taking a risk is worth it. MCTS_v2 agent on the other hand, keeps all moves in the tree but weights

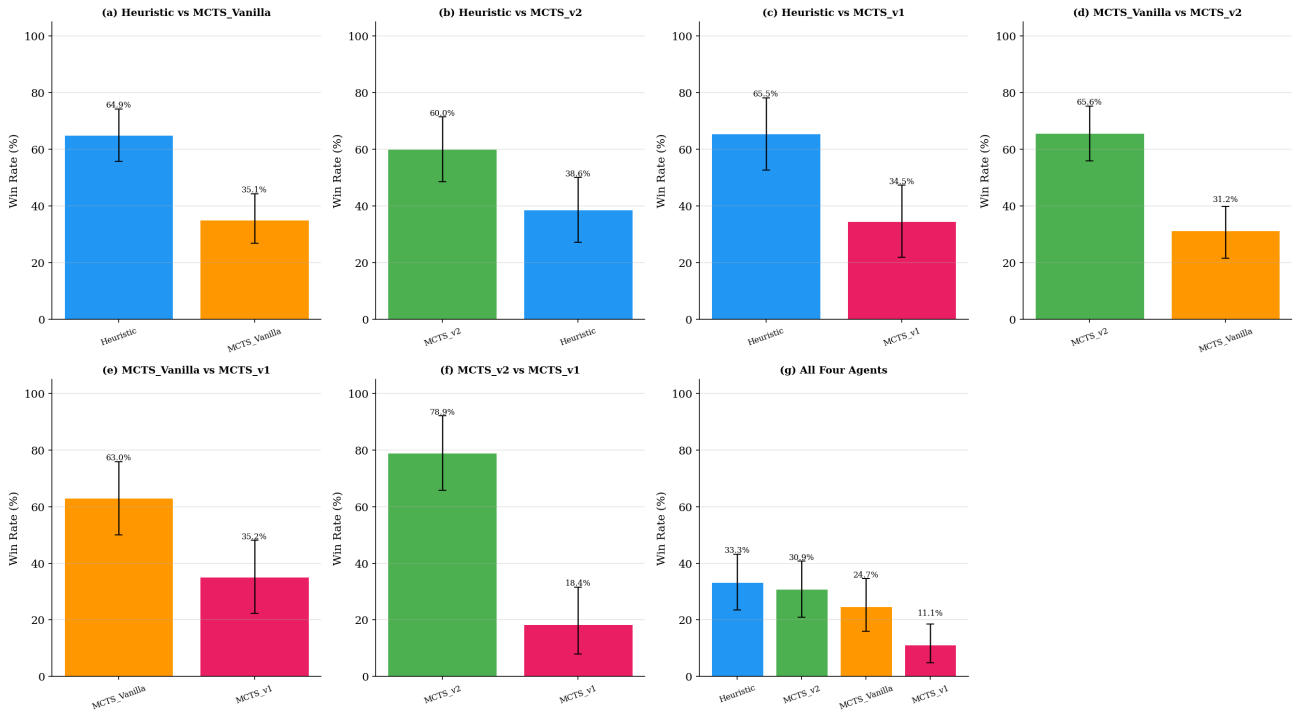

Figure 5. Pairwise and multi-agent win percentages across all seven experimental configurations.

dangerous ones negatively through the heuristic and lets the visit counts decide. This produces more balanced and meaningful decisions in risky situations.

Another important observation that was seen is that heuristic agent performing as the best or close to best agent across different experiments while compared to MCTS variants which is actually the opposite of what is it supposed to be because the heuristic lacks the multi-step look-ahead performed by the MCTS variants. This was because of the strict time constraints we have that doesn't actually allow the MCTS tree's to grow deep enough to utilize its complete potential.

7. Conclusion

This work for building and evaluating different agents for the BattleSnakes game began with the implementation of the heuristic agent which made use of Vornoi flood-filled evaluation. With this as the baseline the next agent Vanilla MCTS was implemented that made use of standard UCB selection with fully random rollouts. With this as base two variants MCTS_v1 and MCTS_v2 with two different logics Rave implementation and a stricter tree pruning logic. But compared to the theoretical expectations the more complex version performed less efficiently compared to its counterpart indicating that in constrained time environments, rollout quality matters more than rollout quantity.

8. Future Works

The agent's adaptability could be improved by opponent modeling where the agent could observe whether its opponent is playing aggressively or safely and exploit that towards planning its moves for a guranteed victory. The search process used can be further refined by progressive widening were the number of children visting each node in a search tree would be limited forcing the agent to first explore more promising branches than random exploration. To further refine the MCTS evaluations, optimizations the agent could prioritizes food in its initial turns, followed by center board dominance with cornering opponents. Making use of neural network instead of the guided rollout policy used in MCTS could also provide better guidance in critical decision making situation.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework, 2019. URL https://arxiv.org/abs/1907.10902.

Hutter, F., Hoos, H., and Leyton-Brown, K. An efficient

approach for assessing hyperparameter importance. In Xing, E. P. and Jebara, T. (eds.), *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pp. 754–762, Bejing, China, 22–24 Jun 2014. PMLR. URL https://proceedings.mlr.press/v32/hutter14.html.

Kocsis, L. and Szepesvári, C. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, pp. 282–293. Springer Berlin Heidelberg, 2006. doi: 10.1007/11871842_29.

Perick, P., Chaslot, G., de Bruin, A., and van den Herik, J. Monte-carlo tree search for the game of tron. In *Proceedings of the 4th International Conference on Agents and Artificial Intelligence*, pp. 352–357. SciTePress, 2012.

Rimmel, A., Teytaud, F., and Teytaud, O. Biasing monte-carlo simulations through rave values. In *Computers and Games*, pp. 59–68. Springer Berlin Heidelberg, 2011. doi: 10.1007/978-3-642-17928-0_6.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Świechowski, M., Godlewski, K., Sawicki, B., and Mańdziuk, J. Monte carlo tree search: a review of recent modifications and applications. *Artificial Intelligence Review*, 56(3):2497–2562, 2022. doi: 10.1007/s10462-022-10228-y.

Winands, M. H. M., Björnsson, Y., and Saito, J.-T. Monte-carlo tree search solver. In *Computers and Games*, pp. 25–36. Springer Berlin Heidelberg, 2008. doi: 10.1007/978-3-540-87608-3_3.

A. Agent Performance and Convergence Analysis

A.1. Tournament Execution and TrueSkill Convergence

To rigorously evaluate our models, we executed a 700-game parallelized tournament. The tournament consisted of seven distinct matchup configurations (six pairwise and one four-agent free-for-all), with 100 matches simulated per configuration. Under this structure, each of the four agents was scheduled to participate in exactly 400 matches.

Figure 6 plots the conservative TrueSkill ratings ($\mu - 3\sigma$) of all agents across their match iterations. After an initial calibration phase, the ratings converge into a stable hierarchy. MCTS_v2 emerges as the dominant strategy, stabilizing at the highest rating (approx. 24), followed closely by the Heuristic agent. The stability of the convergence graph (utilizing a 10-game moving average) indicates the tournament ran for a sufficient duration to confidently establish relative skill levels.

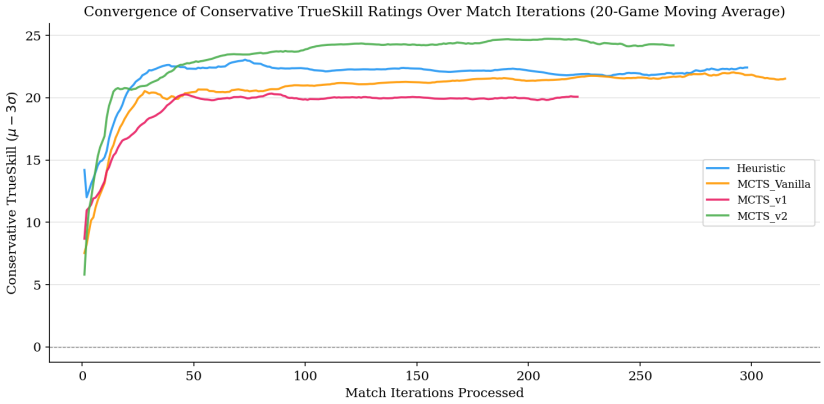

Figure 6. Convergence of TrueSkill ratings ($\mu - 3\sigma$) over match iterations (10-game moving average). All games were played with random seeds for food spawning and agent spawn rotation to ensure fairness.

A.2. Computational Complexity and Timeout Risks

As observed in Figure 6, the convergence lines terminate at varying iteration counts. While all agents were scheduled for 400 matches, none successfully logged all 400. This discrepancy highlights a critical risk in real-time heuristic search: computational cost of complexity.

In the BattleSnakes environment, agents operate under a strict 1000ms response timeout. Our tournament was parallelized across 4 workers, maximizing CPU load. Under this heavy load, agents with highly intensive forward models frequently failed to return a move within the time limit. When an agent times out, the engine aborts its execution, resulting in an unlogged or forfeited match.

Table 1 details the completion and timeout rates for each agent. MCTS_v1, which utilizes both Rapid Action Value Estimation (RAVE) and continuous Voronoi flood-fills during its rollout evaluations, proved too computationally heavy. It suffered a massive 45.0% timeout rate, concluding its run at just 220 matches. MCTS_v2, despite using a complex heuristic, mitigated this by employing strict hierarchical pruning to drastically reduce the breadth of its search tree, achieving a better completion rate. Conversely, the lightweight, purely deterministic Heuristic agent rarely timed out, logging 320 successful matches. This demonstrates that in strict latency environments, a highly complex agent that frequently misses its turn is a severe liability compared to a slightly less optimal, but significantly faster, agent.

A.3. Distribution of Survival Durations

The survival durations (Figure 7) corroborate the TrueSkill hierarchy while further illustrating the impact of timeouts. MCTS_v2 achieves the highest survivability with a median and mean survival time of roughly 130 turns. In contrast, MCTS_v1 struggles the most, frequently expiring before turn 100—largely because its high computational overhead routinely resulted in missed turns, causing the snake to blindly collide with hazards or walls. Outliers clustered at the 300-turn threshold represent matches that terminated via the global turn limit rather than elimination.

Table 1. Match completion and timeout rates per agent across the 400 scheduled tournament games. Higher timeout rates correlate directly with the computational complexity of the agent's search algorithm.

Agent	Scheduled Matches	Completed Matches	Timeouts	Timeout Rate
Heuristic	400	320	80	20.0%
MCTS_Vanilla	400	315	85	21.2%
MCTS_v2	400	265	135	33.8%
MCTS_v1	400	220	180	45.0%

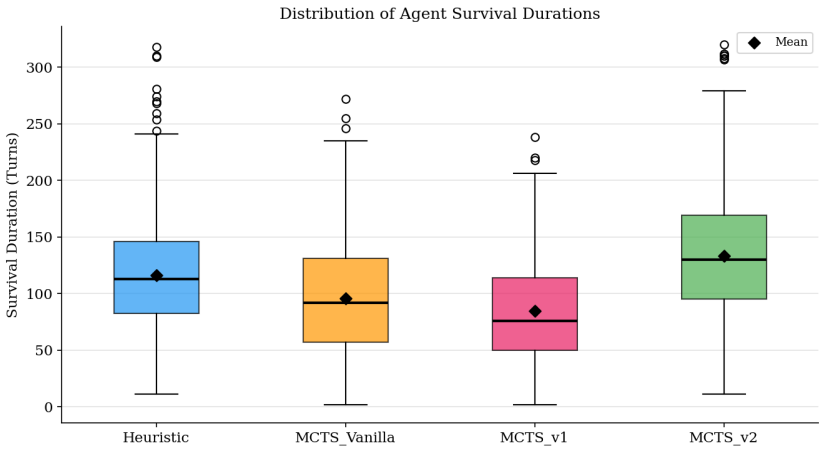

Figure 7. Distribution of agent survival durations. Boxplots indicate the median (solid line) and mean (black diamond) turns survived per agent. Outliers clustered at 300 turns represent matches terminating via global turn limits.

B. Hyperparameter Configurations

To ensure optimal performance and a fair comparison between search methodologies, the MCTS variants were individually tuned. Vanilla MCTS and MCTS_v2 utilized the Optuna framework to maximize a composite objective score, while MCTS_v1 utilized manually bounded environmental parameters optimized for its specific RAVE implementation. The final configurations deployed during the 700-game tournament are documented in Table 2.

Table 2. Final hyperparameter configurations for the deployed MCTS agents.

Hyperparameter	Vanilla MCTS	MCTS_v1 (RAVE)	MCTS_v2 (Pruning)
Exploration Constant (C)	2.207	1.997	1.452
Search Depth Limit	13	21	6
Rollout Depth Limit	N/A	20	N/A
Progressive Bias Weight (W_{PB})	N/A	17.215	10.805
RAVE Equivalence (K)	N/A	1054.73	N/A
Early Game Target Length	N/A	N/A	15

Notably, MCTS_v2 favors a significantly lower exploration constant ($C = 1.452$) and depth limit (6) compared to the Vanilla and v1 iterations. Because MCTS_v2 utilizes hierarchical pruning to strictly isolate safe moves prior to expansion, it benefits primarily from exploiting known safe trajectories rather than exploring broadly, allowing it to perform well even at shallower search horizons.